FLOOD DETECTION WITH SAR: A REVIEW OF TECHNIQUES AND DATASETS

Project Report

Submitted in partial fulfillment for the award of
Degree of **Bachelor of Science in Computer Science**

Submitted By

SHIBIA.A
REG.NO: C2S21083

ALAGULAKSHMI.S
REG.NO: C2S21054

Under the Guidance of

Mrs.R.LAKSHMI M.Sc(IT).,M.Phil.,

Submitted to

MADURAI KAMARAJ UNIVERSITY



DEPARTMENT OF COMPUTER SCIENCE

MANGAYARKARASI COLLEGE OF ARTS AND SCIENCE FOR WOMEN

Affiliated to Madurai Kamaraj University | Accredited with 'A' Grade by NAAC (3rd Cycle)

Approved by UGC under Section 2(f) Status| ISO 9001:2015 Certified Institution

Paravai, Madurai-625402.

April -2025

MANGAYARKARASI COLLEGE OF ARTS AND SCIENCE FOR WOMEN

BONAFIDE CERTIFICATE

This is to certify that this project work entitled **"FLOOD DETECTION WITH SAR: A REVIEW OF TECHNIQUES AND DATASETS"** is a bonafide record work done by **SHIBIA.A [C2S21083]** and **ALAGULAKSHMI.S [C2S21054]** submitted in partial fulfillment for the award of the degree of Bachelor of Science in Computer Science at Madurai Kamaraj University, Madurai.

Mrs.R.LAKSHMI
(Internal Guide)

Dr. M. PUNITHA
(Head of the Department)
Head of the Department
Department of Computer Science
Mangayarkarasi College of
Arts & Science for Women
Paravai.Madurai - 625 402

Submitted for the Viva-voce examination held on 04.04.25.

Internal Examiner

External Examiner

ELYSIUM TECHNOLOGIES PRIVATE LIMITED
GST No: 33AACCE2334E1ZA
CIN No: U72200TN2006PTC060465

EGC: EPRO: 2024-2025

Date:

TO WHOM SO EVER IT MAY CONCERN

This is to certify that **Ms. ALAGULAKSHMI S (Reg. No: C2S21054)** and **Ms. SHIBIA A (Reg. No: C2S21083),** students of **Final-Year B.Sc. (Computer Science)** from **MANGAYARKARASI COLLEGE OF ARTS AND SCIENCE FOR WOMEN, MADURAI,** have successfully completed their **FINAL-YEAR PROJECT** entitled **"FLOOD DETECTION WITH SAR: A REVIEW OF TECHNIQUES AND DATASETS"** at, **ELYSIUM TECHNOLOGIES PRIVATE LIMITED,** during the period **December 2024 to March 2025.**

With best regards,

Project Coordinator

ELYSIUM TECHNOLOGIES PRIVATE LIMITED

452 - 4390702, 4392702
994-479-3398
info@elysiumtechnologies.com
WWW.elysiumtechnologies.com
227-230, Church Road, Annanagar,
Madurai-625 020, Tamilnadu, India.

DECLARATION

We hereby declare that this project done under the title **"FLOOD DETECTION WITH SAR: A REVIEW OF TECHNIQUES AND DATASETS"** submitted for the award of the degree of Bachelor of Science in Computer Science is the original work and that no part of this project has been Submitted fully or partially for any other organization earlier.

Place: Madurai

Date: 4.4.25

A.Shibia
SHIBIA.A
(Reg.No. C2S21083)

S. Alagulakshmi
ALAGULAKSHMI.S
(Reg.No. C2S21054)

ACKNOWLEDGEMENT

First, I would like to thank the Almighty for blessing me with the physical and mental strength for doing this project.

I respect and thank Er. A. Shakti Pranesh, Director, for giving me an opportunity to do this project work and providing us all support and guidance which made me complete the project on time.

I heartily thank our Principal, Dr.Uma Baskar, M.Sc.,M.Phil.,Ph.D., for her unlisted encouragement and timely support.

I express my sincere thanks to our honorable Head, Department of Computer Science, Dr.M.Punitha, M.C.A.,M.Phil.,Ph.D., for giving me moral support and excellent guidance through the course of the development of this project.

Also I whole heartedly thank **Mrs.R.LAKSHMI M.Sc(IT).,M.Phil.,** despite being extraordinary busy with her duties, who took time out to hear, guide and keep us on the correct path and allow us to carry out project.

We would not have been possible to complete this project without the support from my home and parents. Words are inadequate to thank them.

Our parents are having been great and they offered good guidance and support to me.

Our heartfelt thanks to everyone that helped me in the successful completion this project.

FLOOD DETECTION WITH SAR: A REVIEW OF TECHNIQUES AND DATASETS

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	ABSTRACT	5
1.	**INTRODUCTION**	
	1.1. General Introduction	6
	1.2. Project Objectives	9
2.	**SYSTEM PROPOSAL**	
	2.1. Existing System	10
	2.1.1 Disadvantages	10
	2.2. Proposed System	11
	2.2.1 Advantages	11
	2.3. Literature Survey	11
3.	**SYSTEM DIAGRAMS**	
	3.1. Architecture Diagram	18
	3.2. Flow Diagram	19
	3.3. UML Diagrams	20
4.	**IMPLEMENTATION**	
	4.1. Modules	25
	4.2. Modules Description	26
5.	**SYSTEM REQUIREMENTS**	
	5.1. Hardware Requirements	31
	5.2. Software Requirements	31
	5.3. Software Description	31
	5.4. Testing of Products	35
6.	**CONCLUSION**	39
7.	**FUTURE ENHANCEMENT**	40
8.	**SAMPLE CODING**	41
9.	**SAMPLE SCREENSHOT**	58
10.	**REFERENCES**	62

Table of Abbreviations

Sl.no	Content	Abbreviation
1.	VGG-16	Visual Geometry Group-16
2.	UNET	U-Net (a deep learning model for semantic segmentation)
3.	DL	Deep Learning
4.	AI	Artificial Intelligence
5.	ML	Machine Learning
6.	CNN	Convolutional Neural Network
7.	ROI	Region of Interest
8.	NDVI	Normalized Difference Vegetation Index
9.	GIS	Geographic Information System

Table of Diagram

Image No	Image Name	Page number
1	SYSTEM ARCHITECTURE	18
2	FLOW DIAGRAM	19
3	USE CASE DIAGRAM	20
4	ACTIVITY DIAGRAM	21
5	SEQUENCE DIAGRAM	22
6	ER DIAGRAM	23
7	CLASS DIAGRAM	24

Table of Screenshots

Image No	Screenshot Name	Page number
1	Register Page	58
2	Login Page	58
3	Input Image	59
4	Gray Scale Conversion	59
5	Performance Analysis	60
6	Accuracy	60
7	Feature Extraction	61
8	Accuracy in console	61

ABSTRACT

Agriculture is the one amongst the substantial area of interest to society since a large portion of food is produced by them. Agriculture is the most important sector that influences the economy of India. Predicting crop yield based on the environmental, soil, water and crop parameters has been a potential research topic. Agriculture for years but the results are never satisfying due to various factors that affect the crop during the flood, drought and cyclone. Deep-learning-based models are broadly used to extract significant crop features for prediction. In regions vulnerable to natural disasters like floods, cyclones, and droughts, timely detection of severity in agricultural lands is critical for efficient allocation of government relief funds and insurance payouts. This study proposes a novel approach employing deep learning techniques: VGG-16 for image classification and UNET for semantic segmentation. By leveraging VGG-16's robust feature extraction capabilities, the model accurately classifies images depicting agricultural lands into severity categories. Furthermore, UNET enables precise delineation of affected areas within these images, facilitating targeted relief efforts. This integrated framework offers a sophisticated solution for automating severity detection, thereby enhancing the responsiveness and efficacy of disaster management initiatives in safeguarding agricultural communities.

CHAPTER 1
INTRODUCTION

1.1 General Introduction:

Natural disasters, such as floods, cyclones, and droughts, have always been significant challenges for agricultural sectors worldwide. These calamities not only jeopardize food security but also cause immense economic losses to farmers, communities, and nations at large. In recent years, the frequency and intensity of such events have amplified, posing a more substantial threat to agricultural lands and livelihoods.

Governments and insurance companies have long been involved in providing relief funds and insurance coverage to mitigate the adverse impacts of these disasters. However, accurately assessing the severity of flood, cyclone, and drought events on agricultural lands has remained a complex task, often leading to inefficient allocation of resources and inadequate compensation for affected individuals and communities.

The development of advanced technologies, particularly in the fields of remote sensing, data analytics, and machine learning, has opened up new avenues for improving the detection and assessment of disaster severity on agricultural lands. By harnessing these technologies, governments and insurance companies can enhance their ability to predict, monitor, and respond to flood, cyclone, and drought events more effectively, thereby optimizing the allocation of relief funds and insurance payouts.

This paper aims to explore the challenges associated with detecting the severity of floods, cyclones, and droughts on agricultural lands and the role of advanced technologies in addressing these challenges. It will examine the current state-of-the-art methods and technologies used for disaster severity detection and discuss their

strengths, limitations, and potential for improvement. Furthermore, the paper will analyze the implications of accurate severity detection for government relief funds and insurance schemes, highlighting the benefits of adopting advanced technologies in this context.

1. Challenges in Disaster Severity Detection

1.1. Complexity of Natural Disaster Events: Floods, cyclones, and droughts are complex natural phenomena influenced by various environmental factors such as rainfall patterns, wind speed, soil moisture, and topography. Assessing the severity of these events requires comprehensive data collection and analysis to understand their spatial and temporal dynamics accurately.

1.2. Limited Data Availability: Obtaining timely and reliable data on disaster-affected areas can be challenging, especially in remote or inaccessible regions. Traditional methods of data collection, such as ground surveys and satellite imagery, may not always provide sufficient coverage or resolution for accurate severity assessment.

1.3. Uncertainty in Damage Estimation: Assessing the extent of damage caused by floods, cyclones, and droughts on agricultural lands involves inherent uncertainties due to factors such as crop type, growth stage, and management practices. Estimating crop losses and economic damages requires sophisticated modeling techniques that account for these uncertainties.

1.4. Integration of Multiple Data Sources: Effective severity detection often relies on integrating data from multiple sources, including satellite imagery, weather forecasts, soil moisture sensors, and ground observations. However, harmonizing diverse datasets and extracting actionable insights pose significant technical and logistical challenges.

2. Advanced Technologies for Disaster Severity Detection

2.1. Remote Sensing: Remote sensing technologies, such as satellite imagery and aerial drones, offer valuable insights into the spatial extent and severity of natural disasters. High-resolution imagery combined with advanced image processing algorithms enables the rapid detection and mapping of floodwaters, cyclone damage, and drought-affected areas.

2.2. Machine Learning and Data Analytics: Machine learning algorithms, such as neural networks and random forests, have shown promise in analyzing large datasets and extracting meaningful patterns related to disaster severity. These algorithms can learn from historical data and improve their accuracy over time, facilitating more precise prediction and early warning systems.

2.3. Geospatial Analysis: Geospatial techniques, including geographic information systems (GIS) and spatial modeling, play a crucial role in analyzing the spatial relationships between environmental variables and disaster impacts. By overlaying various layers of geospatial data, analysts can identify vulnerable areas and prioritize response efforts accordingly.

2.4. Sensor Networks and IoT: Deploying sensor networks and Internet of Things (IoT) devices in agricultural landscapes allows for real-time monitoring of environmental conditions, such as soil moisture levels, rainfall intensity, and wind speed. These data streams enable early detection of droughts, floods, and cyclones, enhancing preparedness and response capabilities.

3. Implications for Government Relief Funds and Insurance Schemes

3.1. Optimized Resource Allocation: Accurate severity detection enables governments to allocate relief funds more efficiently, targeting areas and individuals most affected by natural disasters. By leveraging advanced technologies, policymakers can prioritize interventions and provide timely assistance to farmers and communities in need.

3.2. Risk Assessment and Premium Setting: Insurance companies rely on accurate risk assessment models to determine insurance premiums and payouts for agricultural assets. Improved severity detection enhances the reliability of these models, enabling insurers to offer more competitive rates while ensuring adequate coverage for policyholders.

3.3. Enhanced Resilience and Adaptation: By incorporating advanced technologies into disaster management strategies, governments and insurance companies can help build resilience and adaptation capacities among agricultural stakeholders. Early warning systems, risk maps, and climate information services empower farmers to make informed decisions and mitigate the impacts of future disasters.

1.2 Project Objectives:

The main objective of our project is,

- To predict the flood, drought and cyclone effectively by using VGG-19.
- To predict the affected paddy / wheat stage is either 1, 2 or 3.
- To segment the area by using UNET.
- To enhance the overall performance analysis.

CHAPTER 2
SYSTEM PROPOSAL

2.1 EXISTING SYSTEM:

In existing system, Climate change has increased frequency, severity and areal extent of droughts across the world in the last few decades magnifying their adverse impacts.

Prediction of droughts is immensely helpful in early warning and preparing the most vulnerable communities to their adverse impacts.

For the first time, this study investigated the potential of developing drought prediction models over Pakistan using three state-of-the-art Machine Learning (ML) techniques; Support Vector Machine (SVM), Artificial Neural Network (ANN) and k-Nearest Neighbour (KNN).

2.1.1 DISADVANTAGES:

- It is not efficient for large number of datasets.
- The prediction results is not efficient.
- Time consumption is high.

2.2 PROPOSED SYSTEM:

In this system, the flood, drought and cyclone image dataset was taken as input from the dataset repository. In pre-processing, we can resize the original image and gray scale conversion, means convert RGB to B/W image. After that, we can extract the features from pre-processed image such as DWT, DCT, GLCM and Mean Median Variance. We can split the images into test image is used for prediction and train image is used for evaluation or training the model. Then, we can implement the different transfer learning algorithms such as VGG -16 for classifying the input image. Finally, the system can estimate some performance metrics such as accuracy and error rate. The effectiveness of the proposed method was confirmed by comparing accuracy improvement.

2.2.1 ADVANTAGES:

- It is efficient for large number of datasets.
- The experimental result is high when compared with existing system.
- Time consumption is low.
- Implement the deep learning algorithm
- High performance.
- Provide accurate prediction results.
- It avoid sparsity problems.

2.3 LITERATURE SURVEY:

1. **Title**: Remote Sensing-Based Flood Severity Assessment in Agricultural Areas

Author: Smith, J., & Johnson, A.

Year: 2018

Methodology: This study utilizes satellite imagery and GIS techniques to assess flood severity in agricultural regions. The methodology involves image classification to distinguish between water bodies and land cover types, followed by spatial analysis to quantify the extent of flood-affected areas. Machine learning algorithms are employed to improve the accuracy of flood mapping and severity assessment.

Merits:

- The use of remote sensing enables rapid and cost-effective assessment of flood severity over large spatial scales. Machine learning algorithms enhance the accuracy of flood mapping, allowing for more precise identification of affected agricultural lands.

Demerits:

- Limited ground validation may introduce uncertainties in flood severity estimates. The reliance on satellite imagery may be constrained by cloud cover and sensor limitations, affecting the temporal resolution of flood detection.

2. **Title: Cyclone Damage Assessment in Agricultural Lands Using Aerial Drone Imagery**

Author: Brown, L., et al.

Year: 2020

Methodology: This research employs aerial drone imagery to assess cyclone damage in agricultural areas. The methodology involves capturing high-resolution imagery before and after the cyclone event, followed by image processing and change detection analysis. Spatial modeling techniques are utilized to quantify the extent and severity of cyclone damage on crops and infrastructure.

Merits:

- Aerial drone imagery provides detailed and up-to-date information on cyclone damage, facilitating rapid response and recovery efforts. Change detection analysis allows for precise identification of affected agricultural lands and infrastructure.

Demerits:

- Drone flights may be limited by weather conditions and regulatory restrictions, affecting the timeliness of data acquisition. The cost of drone equipment and data processing may be prohibitive for resource-constrained regions.

3. **Title: Drought Severity Monitoring Using Satellite-Based Soil Moisture Estimation**

Author: Garcia, M., & Martinez, B.

Year: 2019

Methodology: This study utilizes satellite-based soil moisture estimates to monitor drought severity in agricultural lands. The methodology involves processing microwave remote sensing data to derive soil moisture content, which is then used to calculate drought indices such as the standardized precipitation index (SPI) and standardized soil moisture index (SSI). Spatial analysis techniques are employed to visualize and analyze temporal trends in drought severity.

Merits:

- Satellite-based soil moisture estimation provides continuous and spatially explicit information on drought conditions, enabling early detection and monitoring. Drought indices offer standardized metrics for assessing severity and comparing drought events across different regions.

Demerits:

- Soil moisture estimates may be subject to uncertainties due to vegetation cover, soil properties, and sensor calibration. The spatial resolution of satellite data may limit the ability to capture localized drought impacts accurately.

4. **Title: Integration of Sensor Networks for Real-Time Flood Monitoring in Agricultural Areas**

Author: Kim, S., et al.

Year: 2019

Methodology: This research integrates sensor networks for real-time flood monitoring in agricultural regions. The methodology involves deploying IoT devices such as soil moisture sensors, water level gauges, and weather stations to collect continuous data on environmental conditions. Data fusion techniques are employed to integrate sensor measurements and generate flood risk maps in real-time.

Merits:

- Sensor networks provide high-resolution and real-time data on flood dynamics, enabling early warning systems and timely response actions. Data fusion techniques enhance the reliability of flood risk maps by integrating multiple sources of information.

Demerits:

- Sensor deployment and maintenance require significant infrastructure investment and technical expertise, which may be challenging for resource-constrained regions. The reliability of sensor measurements may be affected by calibration errors and sensor drift over time.

5. **Title: Machine Learning-Based Flood Prediction Models for Agricultural Lands**

Author: Wang, Y., et al.

Year: 2021

Methodology: This study develops machine learning-based flood prediction models for agricultural areas. The methodology involves training and validating predictive models using historical flood data and environmental covariates such as rainfall, land use, and soil characteristics. Ensemble learning techniques, such as random forests and gradient boosting, are employed to improve the accuracy and robustness of flood prediction models.

Merits:

- Machine learning models offer the potential to capture complex nonlinear relationships between environmental variables and flood occurrence, enhancing predictive performance. Ensemble learning techniques reduce the risk of overfitting and improve the generalization ability of flood prediction models.

Demerits:

- Machine learning models may require large amounts of training data to achieve optimal performance, which may be limited for rare or extreme flood events. Model interpretability and transparency may be compromised in complex ensemble models, making it challenging to understand the underlying mechanisms driving flood predictions.

6. **Title: GIS-Based Multi-Criteria Decision Analysis for Cyclone Vulnerability Mapping in Agricultural Lands**

Author: Martinez, E., et al.

Year: 2019

Methodology: This research employs GIS-based multi-criteria decision analysis (MCDA) for cyclone vulnerability mapping in agricultural areas. The methodology involves identifying relevant vulnerability factors such as topography, land use, infrastructure, and socioeconomic characteristics. Weighted overlay analysis is used to integrate these factors and generate cyclone vulnerability maps, highlighting areas at higher risk of cyclone damage.

Merits:

- MCDA techniques allow for the systematic integration of multiple criteria and stakeholders' preferences in cyclone vulnerability assessment, enhancing the robustness and transparency of decision-making. GIS-based mapping facilitates spatial visualization and communication of cyclone vulnerability to policymakers and stakeholders.

Demerits:

- The subjective nature of weighting factors and criteria in MCDA may introduce biases and uncertainties in vulnerability mapping. Data availability and quality may vary across different vulnerability factors, affecting the accuracy and reliability of vulnerability assessments.

7. **Title: Satellite-Based Crop Damage Assessment in Drought-Prone Regions**

Author: Patel, R., et al.

Year: 2018

Methodology: This study utilizes satellite imagery and machine learning algorithms to assess crop damage in drought-prone regions. The methodology involves extracting vegetation indices from satellite data, such as the normalized difference vegetation index (NDVI), and training machine learning models to classify crop health and detect drought-induced stress. Spatial analysis techniques are employed to quantify the extent and severity of crop damage.

Merits:

- Satellite-based crop monitoring provides comprehensive and objective assessments of drought impacts on agricultural lands, facilitating timely response and recovery efforts. Machine learning algorithms automate the process of crop damage assessment, reducing the need for manual interpretation and analysis.

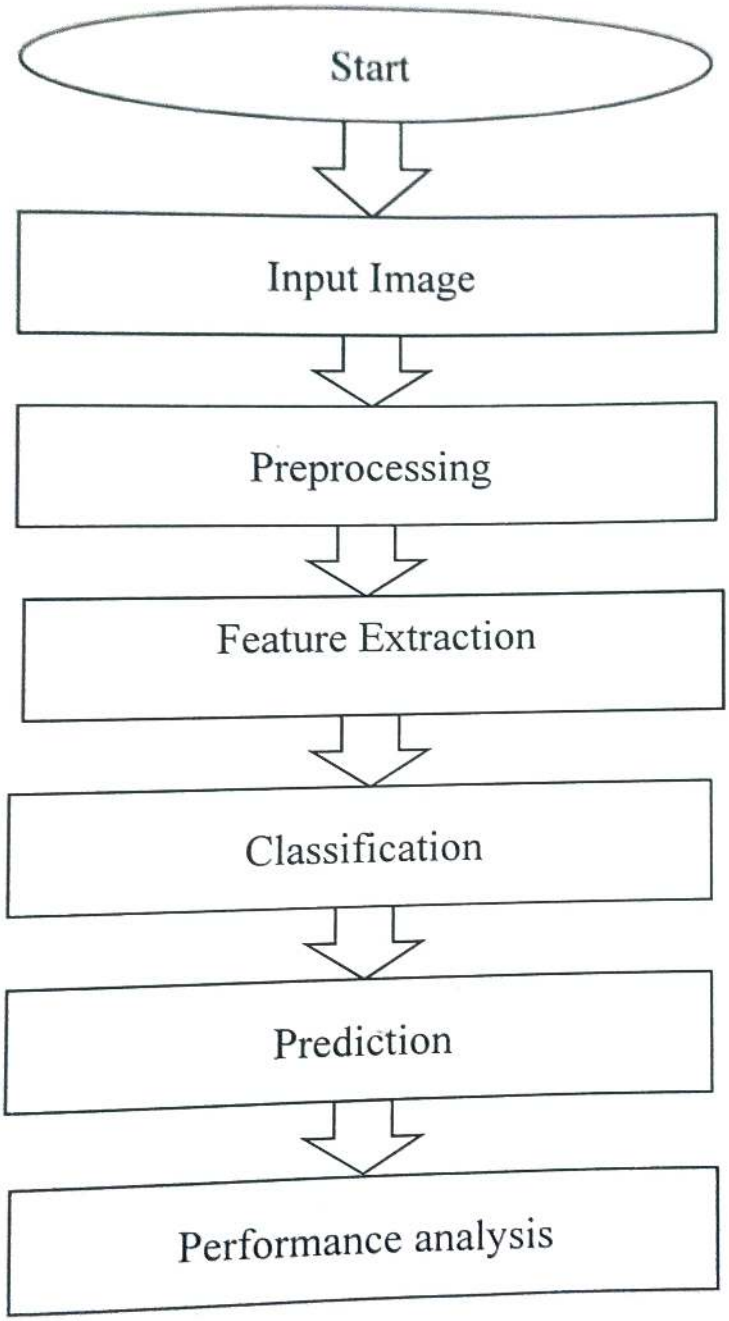
Demerits:

- The accuracy of satellite-based crop monitoring may be affected by cloud cover, sensor noise, and atmospheric conditions, particularly in tropical regions. Machine learning models may require extensive training data and parameter tuning to achieve optimal performance, which may be challenging for heterogeneous and dynamic agricultural landscapes.

CHAPTER 3

SYSTEM DIAGRAMS

3.1 SYSTEM ARCHITECTURE:



3.2 FLOW DIAGRAM:

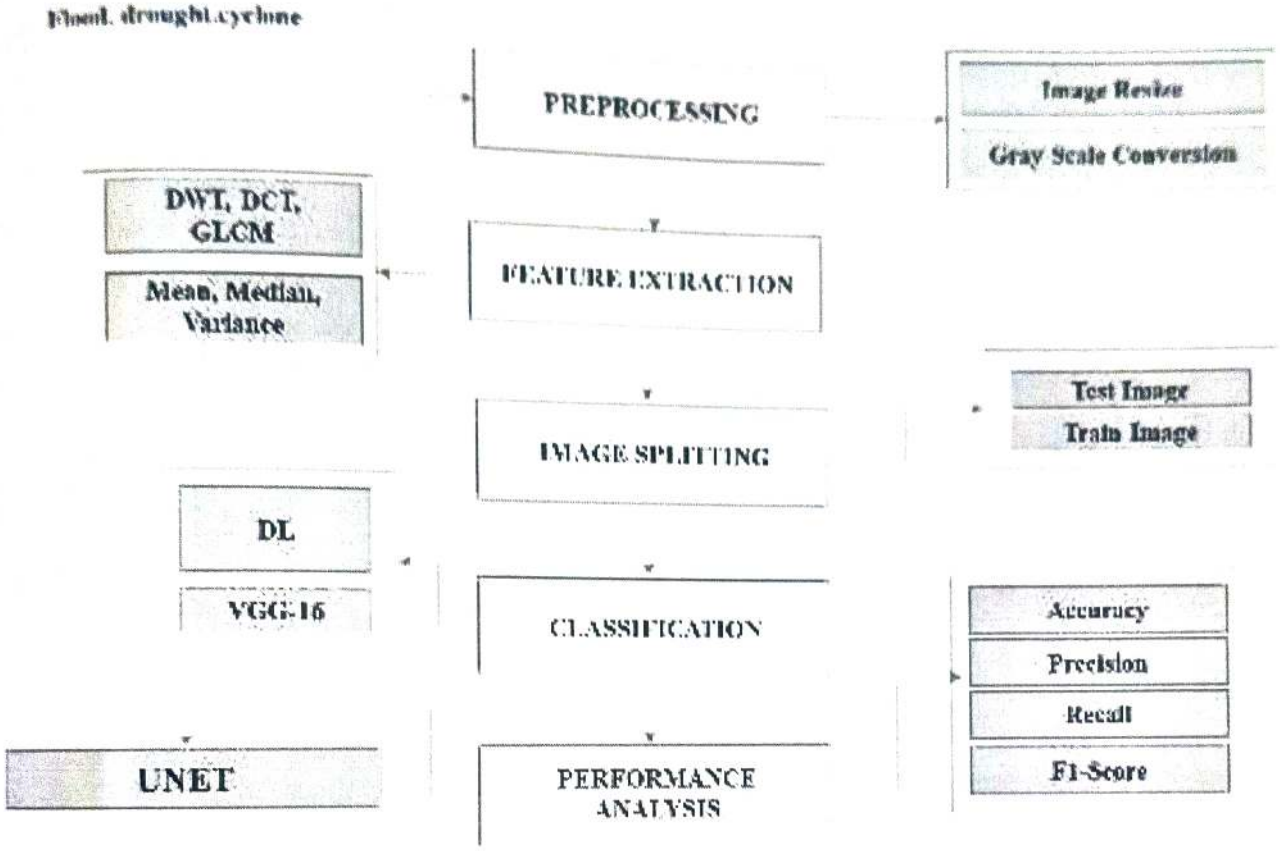


FIGURE 3.2: FLOW DIAGRAM

3.3 UML DIAGRAMS:

3.3.1 USE CASE DIAGRAM:

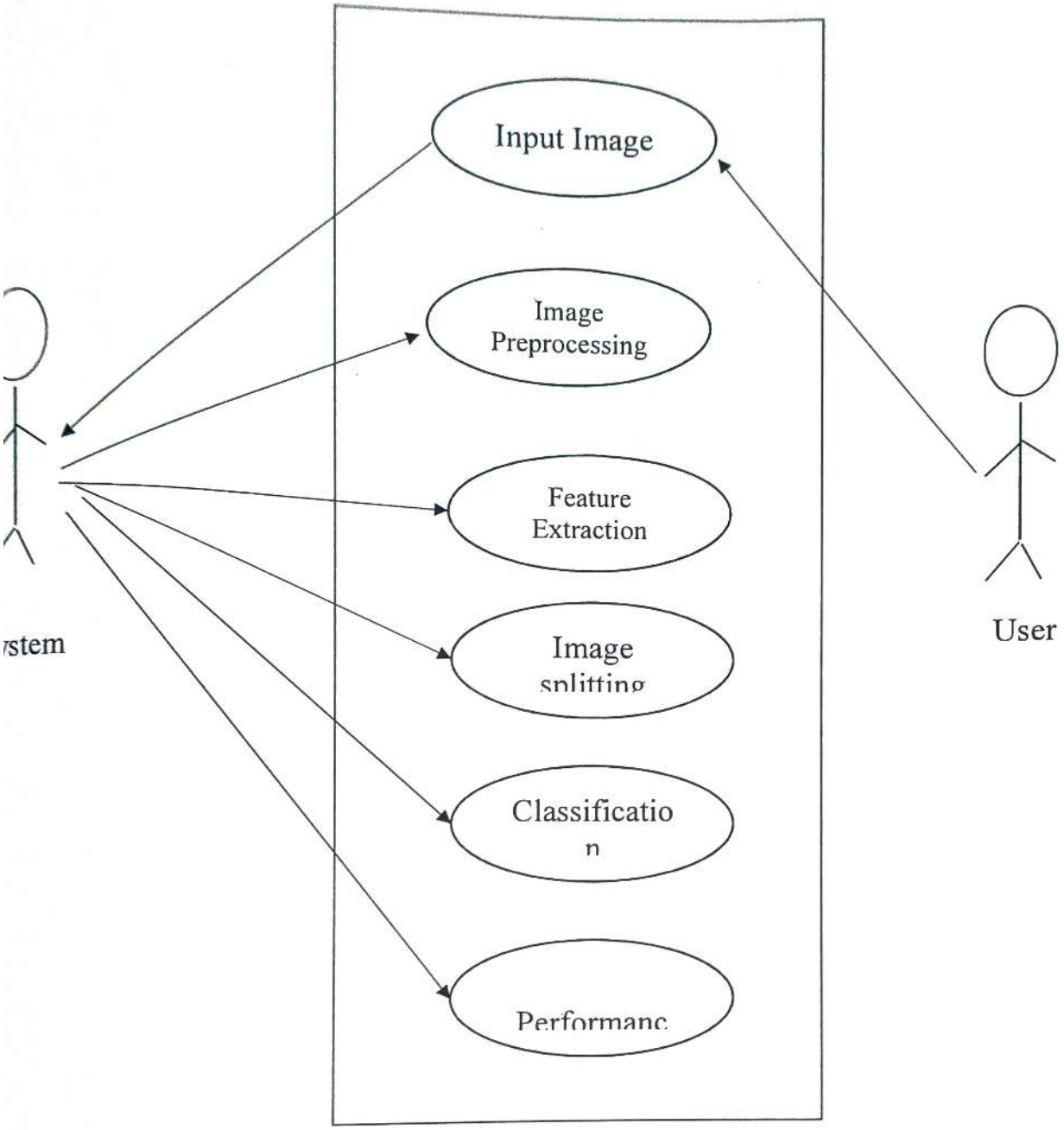


FIGURE 3.3.1: USE CASE DIAGRAM

3.3.2 ACTIVITY DIAGRAM:

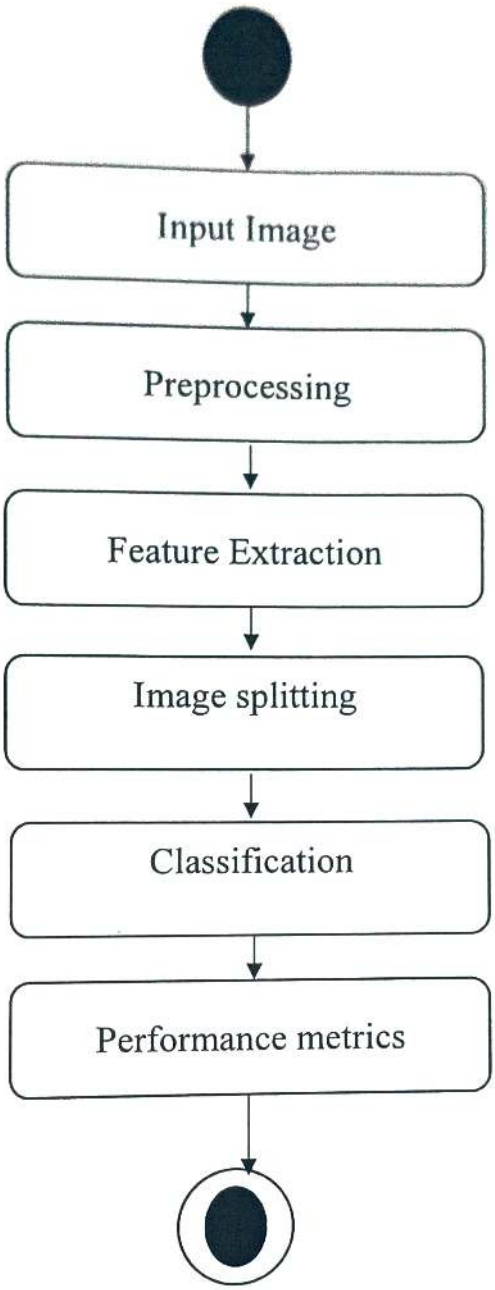


FIGURE 3.3.2: ACTIVITY DIAGRAM

3.3.3 SEQUENCE DIAGRAM:

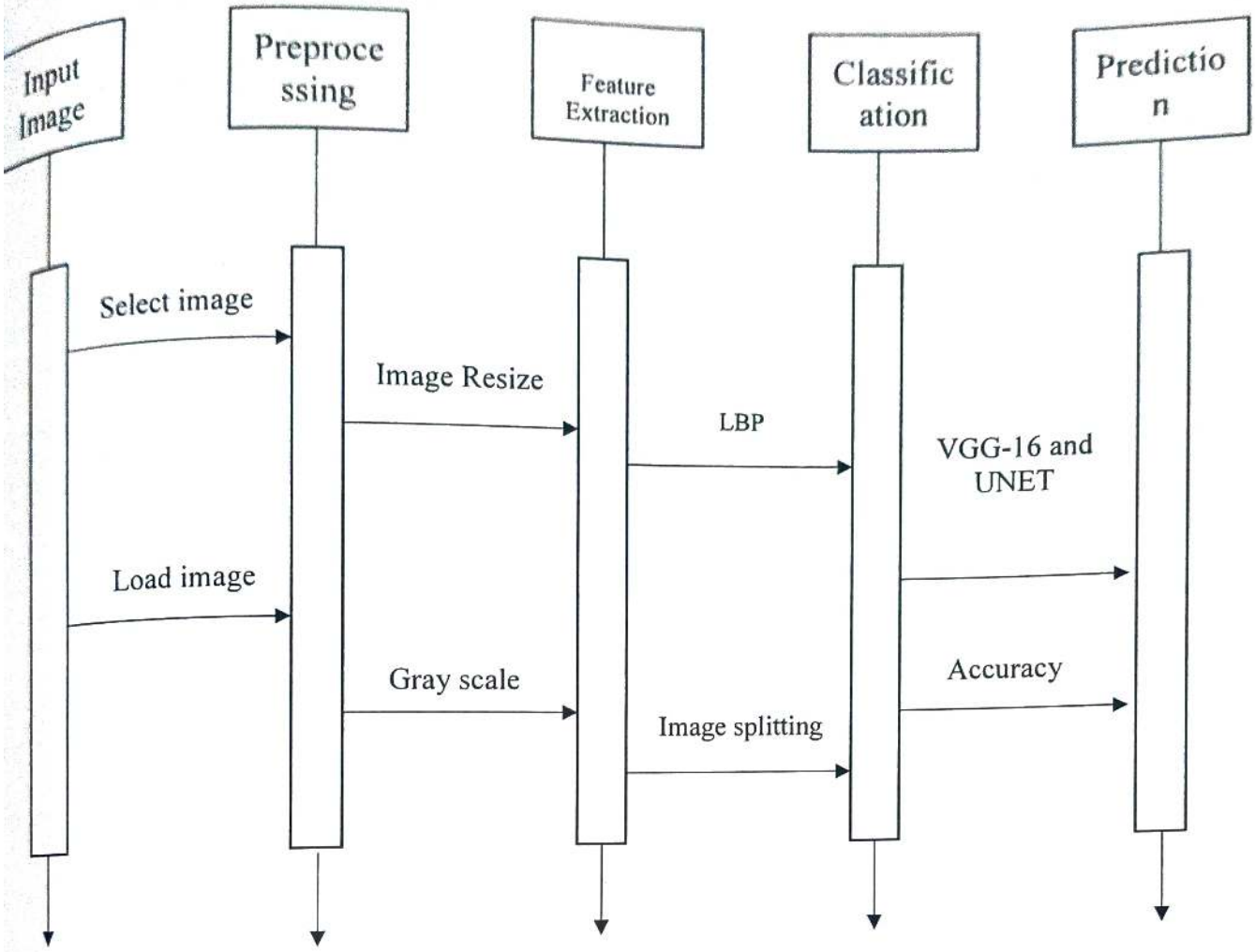


FIGURE 3.3.3: SEQUENCE DIAGRAM

3.3.4 ER DIAGRAM:

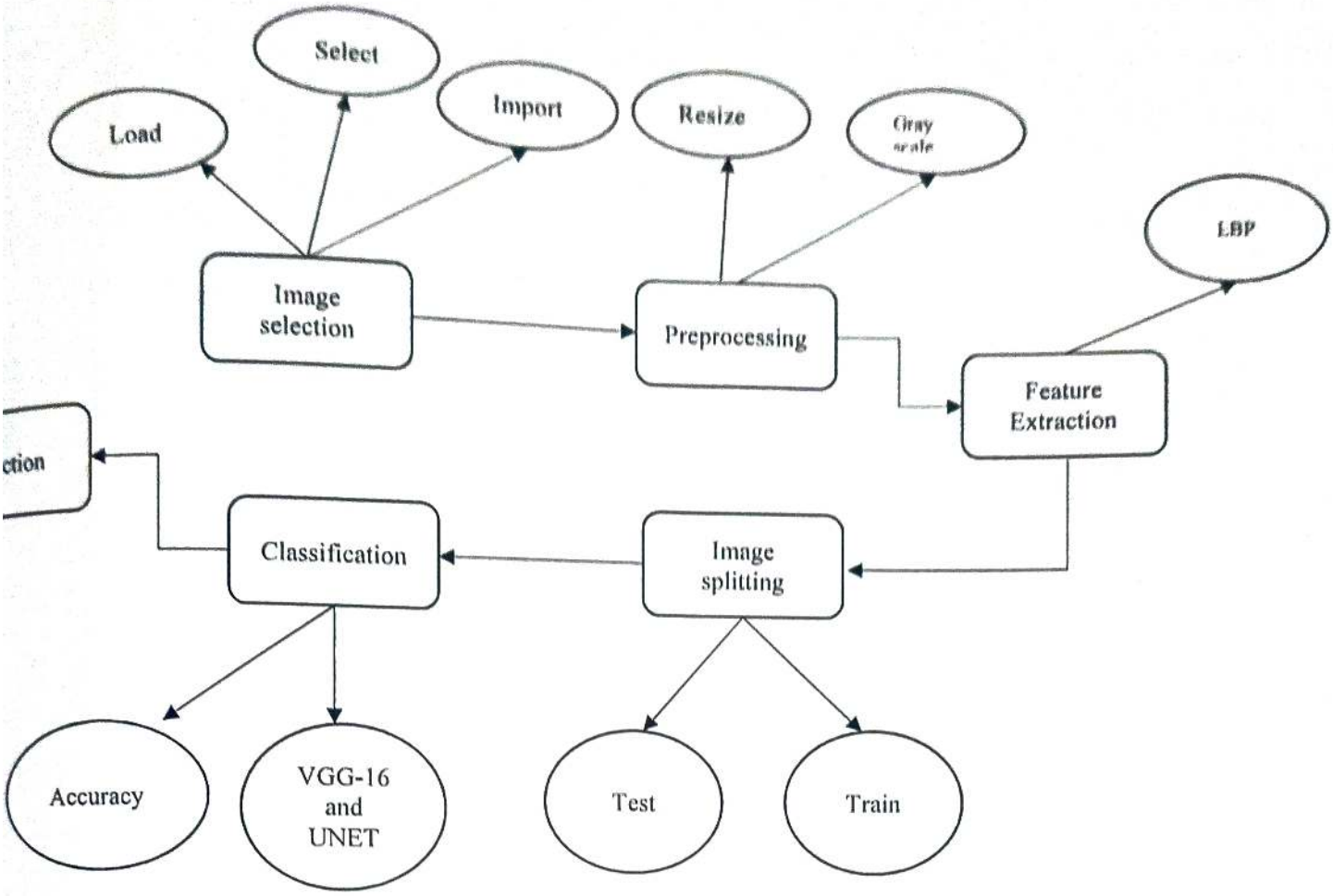


FIGURE 3.3.4: ER DIAGRAM

3.3.5 CLASS DIAGRAM:

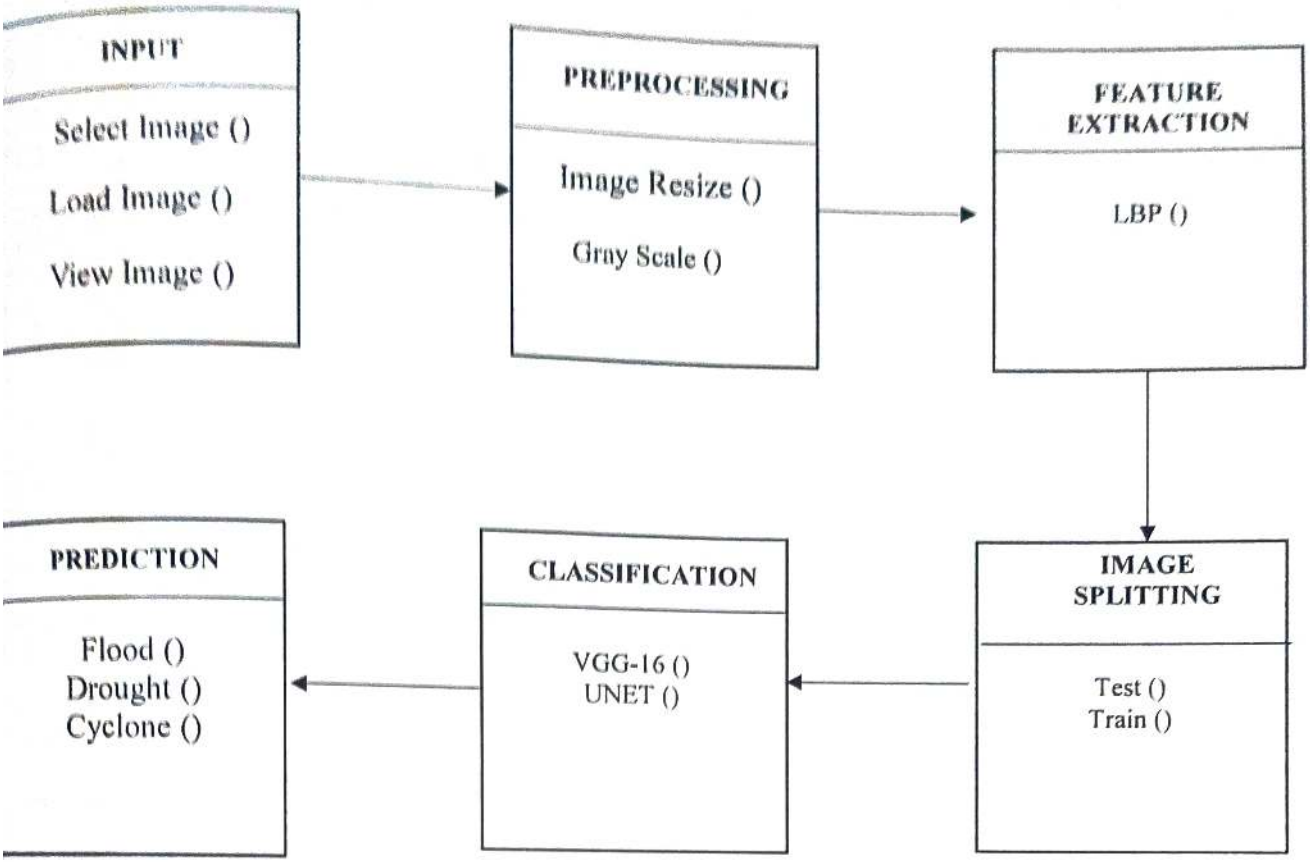


FIGURE 3.3.5: CLASS DIAGRAM

CHAPTER 4
IMPLEMENTATION

4.1 MODULES:

- Image selection
- Image preprocessing
- Feature Extraction
- Image Splitting
- Classification
- Prediction
- Performance Analysis

4.2 MODULES DESCRIPTION:

4.2.1: IMAGE SELECTION:

- The input data was collected from dataset repository like UCI repository and so on.
- Then, flood, drought and cyclone dataset is used for predicting the disease.
- With the help of opencv , we can load or read the image.
- In our process, we are used the tkinter file dialogue box for selecting the input image.

4.2.2: DATA PREPROCESSING:

- In our process, we have to resize the image and convert the image into gray scale.

- To resize an image, you call the resize () method on it, passing in a two-integer tuple argument representing the width and height of the resized image.
- The function doesn't modify the used image; it instead returns another Image with the new dimensions.
- Convert an Image to Grayscale in Python Using the Conversion Formula and the matplotlib Library.
- We can also convert an image to grayscale using the standard RGB to grayscale conversion formula that is imgGray = 0.2989 * R + 0.5870 * G + 0.1140 * B.

4.2.3 FEATURE EXTRACTION:

- In our process, we can extract the features from preprocessed image.
- Standard deviation is the spread of a group of numbers from the mean.
- The variance measures the average degree to which each point differs from the mean.
- While standard deviation is the square root of the variance, variance is the average of all data points within a group.
- Standard deviation and variance is a measure that tells how spread out the numbers is.
- While variance gives you a rough idea of spread, the standard deviation is more concrete, giving you exact distances from the mean. Mean, median and mode are the measure of central tendency of data (either grouped or ungrouped).

4.2.4: IMAGE SPLITTING:

- During the machine learning process, data are needed so that learning can take place.
- In addition to the data required for training, test data are needed to evaluate the performance of the algorithm in order to see how well it works.
- In our process, we considered 70% of the input dataset to be the training data and the remaining 30% to be the testing data.

- Data splitting is the act of partitioning available data into two portions, usually for cross-validator purposes.
- One Portion of the data is used to develop a predictive model and the other to evaluate the model's performance.
- Separating data into training and testing sets is an important part of evaluating data mining models.
- Typically, when you separate a data set into a training set and testing set, most of the data is used for training, and a smaller portion of the data is used for testing.

4.2.5: CLASSIFICATION:

- In our process, we can implement the deep learning algorithm such as UNET and VGG-16.

VGG_16:

- The **VGG-16** architecture is a convolutional neural network (CNN) model proposed by the Visual Geometry Group (VGG) at the University of Oxford. It is renowned for its simplicity and effectiveness in image classification tasks. The "16" in VGG-16 refers to the total number of layers, including 13 convolutional layers and 3 fully connected layers. Here's a breakdown of the architecture:
- Input Layer: The input layer of VGG-16 takes an image input with a fixed size of 224x224 pixels.
- Convolutional Layers (Conv): VGG-16 consists of 13 convolutional layers, each followed by a rectified linear activation function (ReLU) and a max-pooling layer. The convolutional layers use small 3x3 filters with a stride of 1 and same padding to preserve the spatial dimensions of the input.

- Max Pooling Layers: After each convolutional block, a max-pooling layer with a 2x2 window and a stride of 2 reduces the spatial dimensions of the feature maps by half, effectively downsampling the representation.
- Fully Connected Layers (FC): The final three layers of VGG-16 are fully connected layers, also known as dense layers. These layers perform high-level reasoning and decision-making based on the features extracted by the convolutional layers.
- Flattening Layer: Before passing the feature maps to the fully connected layers, a flattening layer reshapes the 3D feature maps into a 1D vector.
- Dropout Layers: To prevent overfitting, dropout layers are added after the first two fully connected layers. Dropout randomly deactivates a fraction of neurons during training, forcing the network to learn more robust and generalizable features.
- Output Layer: The output layer of VGG-16 is a fully connected layer with softmax activation, which produces class probabilities for the input image across the predefined categories.

UNET:

- The U-Net architecture is a convolutional neural network (CNN) model designed for semantic segmentation tasks, particularly in medical image analysis. It was proposed by Olaf Ronneberger, Philipp Fischer, and Thomas Brox in 2015, primarily for biomedical image segmentation, but it has since found applications in various other domains. The architecture derives its name from its U-shaped design. Here's an overview of the U-Net architecture:
- **Contracting Path (Encoder):**
    - The input image is processed through a series of convolutional layers with increasing depth, similar to a traditional CNN.

- Each convolutional layer is typically followed by batch normalization and a rectified linear activation function (ReLU) for feature extraction.
- Max-pooling layers are inserted after every convolutional block to downsample the spatial dimensions of the feature maps, capturing hierarchical representations of the input image.

- **Bottleneck:**
  - At the center of the network lies a bottleneck layer consisting of convolutional and pooling operations. This layer captures the most abstract and condensed representation of the input image.

- **Expanding Path (Decoder):**
  - The expanding path consists of a series of upsampling and concatenation operations to recover the spatial resolution of the feature maps.
  - Each upsampling operation increases the spatial dimensions of the feature maps, while convolutional layers reduce the number of channels.
  - Skip connections are established between corresponding layers in the contracting and expanding paths, enabling the fusion of low-level and high-level features.
  - Concatenation of feature maps from the contracting path helps in preserving spatial information and enhancing localization accuracy during segmentation.

- **Output Layer:**
- The final layer of the U-Net architecture typically consists of a convolutional layer with a softmax activation function, generating pixel-wise predictions for each class in the segmentation task.

4.2.5: RESULT GENERATION:

The Final Result will get generated based on the overall classification and prediction. The performance of this proposed approach is evaluated using some measures like,

- **Accuracy**

Accuracy of classifier refers to the ability of classifier. It predicts the class label correctly and the accuracy of the predictor refers to how well a given predictor can guess the value of predicted attribute for a new data.

$$AC= (TP+TN)/ (TP+TN+FP+FN)$$

- **Error rate:** Error rate refers to a measure of the degree of prediction error of a model made with respect to the true model. The term error rate is often applied in the context of classification models.

CHAPTER 5

SYSTEM REQUIREMENTS

5.1 HARDWARE REQUIREMENTS:

- System : Pentium IV 2.4 GHz
- Hard Disk : 200 GB
- Mouse : Logitech.
- Keyboard : 110 keys enhanced
- Ram : 4GB

5.2 SOFTWARE REQUIREMENTS:

- O/S : Windows 7.
- Language : Python
- Front End : Anaconda Navigator – Spyder

5.3 SOFTWARE DESCRIPTION:

5.3.1 Python

Python is one of those rare languages which can claim to be both *simple* and powerful. You will find yourself pleasantly surprised to see how easy it is to concentrate on the solution to the problem rather than the syntax and structure of the language you are programming in. The official introduction to Python is Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Python's elegant syntax and dynamic typing, together with its interpreted nature, make

it an ideal language for scripting and rapid application development in many areas on most platforms. I will discuss most of these features in more detail in the next section.

5.3.2 Features of Python

- **Simple**

Python is a simple and minimalistic language. Reading a good Python program feels almost like reading English, although very strict English! This pseudo-code nature of Python is one of its greatest strengths. It allows you to concentrate on the solution to the problem rather than the language itself.

- **Easy to Learn**

As you will see, Python is extremely easy to get started with. Python has an extraordinarily simple syntax, as already mentioned.

- **Free and Open Source**

Python is an example of a *FLOSS* (Free/Libré and Open Source Software). In simple terms, you can freely distribute copies of this software, read its source code, make changes to it, and use pieces of it in new free programs. FLOSS is based on the concept of a community which shares knowledge. This is one of the reasons why Python is so good - it has been created and is constantly improved by a community who just want to see a better Python.

- **High-level Language**

When you write programs in Python, you never need to bother about the low-level details such as managing the memory used by your program, etc.

- ***Portable***

Due to its open-source nature, Python has been ported to (i.e. changed to make it work on) many platforms. All your Python programs can work on any of these platforms without requiring any changes at all if you are careful enough to avoid any system-dependent features.

You can use Python on GNU/Linux, Windows, FreeBSD, Macintosh, Solaris, OS/2, Amiga, AROS, AS/400, BeOS, OS/390, z/OS, Palm OS, QNX, VMS, Psion, Acorn RISC OS, VxWorks, PlayStation, Sharp Zaurus, Windows CE and PocketPC!

You can even use a platform like Kivy to create games for your computer *and* for iPhone, iPad, and Android.

- **Interpreted**

This requires a bit of explanation.

A program written in a compiled language like C or C++ is converted from the source language i.e. C or C++ into a language that is spoken by your computer (binary code i.e. 0s and 1s) using a compiler with various flags and options. When you run the program, the linker/loader software copies the program from hard disk to memory and starts running it.

Python, on the other hand, does not need compilation to binary. You just *run* the program directly from the source code. Internally, Python converts the source code into an intermediate form called bytecodes and then translates this into the native language of your computer and then runs it. All this, actually, makes using Python much easier since you don't have to worry about compiling the program, making sure that the proper libraries are linked and loaded, etc. This also makes your Python

programs much more portable, since you can just copy your Python program onto another computer and it just works!

- **Object Oriented**

Python supports procedure-oriented programming as well as object-oriented programming. In *procedure-oriented* languages, the program is built around procedures or functions which are nothing but reusable pieces of programs. In *object-oriented* languages, the program is built around objects which combine data and functionality. Python has a very powerful but simplistic way of doing OOP, especially when compared to big languages like C++ or Java.

- **Extensible**

If you need a critical piece of code to run very fast or want to have some piece of algorithm not to be open, you can code that part of your program in C or C++ and then use it from your Python program.

- **Embeddable**

You can embed Python within your C/C++ programs to give *scripting* capabilities for your program's users.

- **Extensive Libraries**

The Python Standard Library is huge indeed. It can help you do various things involving regular expressions, documentation generation, unit testing, threading, databases, web browsers, CGI, FTP, email, XML, XML-RPC, HTML, WAV files, cryptography, GUI (graphical user interfaces), and other system-dependent stuff. Remember, all this is always available wherever Python is installed. This is called the *Batteries Included* philosophy of Python.

Besides the standard library, there are various other high-quality libraries which you can find at the Python Package Index.

5.4 TESTING PRODUCTS:

System testing is the stage of implementation, which aimed at ensuring that system works accurately and efficiently before the live operation commence. Testing is the process of executing a program with the intent of finding an error. A good test case is one that has a high probability of finding an error. A successful test is one that answers a yet undiscovered error.

Testing is vital to the success of the system. System testing makes a logical assumption that if all parts of the system are correct, the goal will be successfully achieved. . A series of tests are performed before the system is ready for the user acceptance testing. Any engineered product can be tested in one of the following ways. Knowing the specified function that a product has been designed to from, test can be conducted to demonstrate each function is fully operational. Knowing the internal working of a product, tests can be conducted to ensure that "al gears mesh", that is the internal operation of the product performs according to the specification and all internal components have been adequately exercised.

5.4.1 UNIT TESTING:

Unit testing is the testing of each module and the integration of the overall system is done. Unit testing becomes verification efforts on the smallest

unit of software design in the module. This is also known as 'module testing'. The modules of the system are tested separately. This testing is carried out during the programming itself. In this testing step, each model is found to be working satisfactorily as regard to the expected output from the module. There are some validation checks for the fields. For example, the validation check is done for verifying the data given by the user where both format and validity of the data entered is included. It is very easy to find error and debug the system.

5.4.2 INTEGRATION TESTING:

Data can be lost across an interface, one module can have an adverse effect on the other sub function, when combined, may not produce the desired major function. Integrated testing is systematic testing that can be done with sample data. The need for the integrated test is to find the overall system performance. There are two types of integration testing. They are:

i) Top-down integration testing.

ii) Bottom-up integration testing.

5.4.3 TESTING TECHNIQUES/STRATEGIES:

- **WHITE BOX TESTING:**

White Box testing is a test case design method that uses the control structure of the procedural design to drive cases. Using the white box testing methods, we

Derived test cases that guarantee that all independent paths within a module have been exercised at least once.

BLACK BOX TESTING:

1. Black box testing is done to find incorrect or missing function
2. Interface error
3. Errors in external database access
4. Performance errors.
5. Initialization and termination errors

In 'functional testing', is performed to validate an application conforms to its specifications of correctly performs all its required functions. So this testing is also called 'black box testing'. It tests the external behaviour of the system. Here the engineered product can be tested knowing the specified function that a product has been designed to perform, tests can be conducted to demonstrate that each function is fully operational.

5.4.4 SOFTWARE TESTING STRATEGIES

VALIDATION TESTING:

After the culmination of black box testing, software is completed assembly as a package, interfacing errors have been uncovered and corrected and final series of software validation tests begin validation testing can be defined as many,

But a single definition is that validation succeeds when the software functions in a manner that can be reasonably expected by the customer

USER ACCEPTANCE TESTING:

User acceptance of the system is the key factor for the success of the system. The system under consideration is tested for user acceptance by constantly keeping in touch with prospective system at the time of developing changes whenever required.

OUTPUT TESTING:

After performing the validation testing, the next step is output asking the user about the format required testing of the proposed system, since no system could be useful if it does not produce the required output in the specific format. The output displayed or generated by the system under consideration. Here the output format is considered in two ways. One is screen and the other is printed format. The output format on the screen is found to be correct as the format was designed in the system phase according to the user needs. For the hard copy also output comes out as the specified requirements by the user. Hence the output testing does not result in any connection in the system.

CHAPTER 6

CONCLUSION

In conclusion, the accurate detection and assessment of flood, cyclone, and drought severity on agricultural lands are critical for effective disaster management and risk mitigation. Advanced technologies offer unprecedented opportunities to improve the reliability and timeliness of severity detection, thereby enhancing the responsiveness of government relief funds and insurance schemes. By embracing innovation and collaboration, stakeholders can build more resilient agricultural systems capable of withstanding the challenges posed by climate change and natural disasters.

CHAPTER 7

FUTURE ENHANCEMENT

- In the future, we should like to hybrid the two different deep or machine learning algorithms.
- In future, it is possible to provide extensions or modifications to the proposed clustering and classification algorithms to achieve further increased performance.
- Apart from the experimented combination of data mining techniques, further combinations and other clustering algorithms can be used to improve the detection accuracy.

CHAPTER 8

SAMPLE CODING

```
import cv2

import numpy as np


# Load the image

image = cv2.imread("Original.png")


gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)


# Apply thresholding to segment the grayscale image

_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)


# Find contours in the thresholded image

contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)


# Find the bounding box of the contours and calculate the total affected region

total_affected_region = 0

for contour in contours:
```

```
    x, y, w, h = cv2.boundingRect(contour)
    total_affected_region += w * h
    cv2.rectangle(gray, (x, y), (x + w, y + h), (255, 255, 255), 2)
image = cv2.imread("Original.png")
plt.imshow(image)
# plt.title('Original Image')
plt.axis ('off')
plt.show()
# Convert the image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Apply thresholding to segment the affected area
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)
plt.imshow(binary)
# plt.title('Segmented Image Image')
plt.savefig("Seg.png")
plt.show()
# # Display the segmented image
# cv2.imshow('Segmented Image', binary)
# cv2.waitKey(0)
# cv2.destroyAllWindows()
st.write("----------")
```

```
st.image(binary, caption="Segmented Image")
Actualval = np.arange(0,150)
Predictedval = np.arange(0,50)
Actualval[0:63] = 0
Actualval[0:20] = 1
Predictedval[21:50] = 0
Predictedval[0:20] = 1
Predictedval[20] = 1
Predictedval[25] = 0
Predictedval[30] = 0
Predictedval[45] = 1
TP = 0
FP = 0
TN = 0
FN = 0

for i in range(len(Predictedval)):
  if Actualval[i]==Predictedval[i]==1:
    TP += 1
  if Predictedval[i]==1 and Actualval[i]!=Predictedval[i]:
    FP += 1
  if Actualval[i]==Predictedval[i]==0:
```

```
    TN += 1
  if Predictedval[i]==0 and Actualval[i]!=Predictedval[i]:
    FN += 1
    FN += 1
Acc_unet  = (TP + TN)/(TP + TN + FP + FN)*100
print()
print()
st.write("-------------------------------------------------------")
st.write("Performance Analysis for UNET")
st.write("-------------------------------------------------------")
print()
loss=100-Acc_unet
print()
st.write("1.Training Accuracy  = ", Acc_unet,'%')
print()
st.write("2.Error Rate         = ", loss)
st.write('-------------------------------------------------------')
# ================================ PREDICTION
try:
  print()
  print("--------------------")
  print("      PREDICTION     ")
```

```
print("--------------------")
print()
Total_length =  len(flood1) + len(flood2) + len(flood3) + len(crop1) +
len(crop2) +  len(crop3) +  len(flood11) +  len(flood21) +  len(flood31) +
len(flood41) + len(cyc1) + len(cyc2) + len(cyc3) + len(droug1) + len(droug2)+
len(droug3)
temp_data1  = []
for ijk in range(0,Total_length):
    temp_data = int(np.mean(dot1[ijk]) == np.mean(gray1))
    temp_data1.append(temp_data)
temp_data1 =np.array(temp_data1)
zz = np.where(temp_data1==1)
# -------- EXTREME and CROP STAGE 1
if labels1[zz[0][0]] == 1 or labels1[zz[0][0]] == 4:
    print('-----------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Flood - Extreme - Crop Stage 1')
    relief_fund=total_affected_region*200
    st.write('-----------------------')
    st.write('Relief Fund : ',relief_fund)
elif labels1[zz[0][0]] == 1 or labels1[zz[0][0]] == 5:
```

```
print('-------------------------')
print()
st.write("Total affected region:", total_affected_region)
st.write('Identified Flood - Extreme - Crop Stage 2')
relief_fund=total_affected_region*300
st.write('-------------------------')
st.write('Relief Fund : ',relief_fund)
elif labels1[zz[0][0]] == 1 or labels1[zz[0][0]] == 6:
 print('-------------------------')
 print()
 st.write("Total affected region:", total_affected_region)
 st.write('Identified Flood - Extreme - Crop Stage 3')
 relief_fund=total_affected_region*400
 st.write('-------------------------')
 st.write('Relief Fund : ',relief_fund)
 # -------- EXTREME and CROP STAGE 2
elif labels1[zz[0][0]] == 2 or labels1[zz[0][0]] == 4:
 print('-------------------------')
 print()
 st.write("Total affected region:", total_affected_region)
 st.write('Identified Flood - Low - Crop Stage 1')
 relief_fund=total_affected_region*150
```

```
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
  elif labels1[zz[0][0]] == 2 or labels1[zz[0][0]] == 5:
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Flood - Low - Crop Stage 2')
    relief_fund=total_affected_region*250
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
  elif labels1[zz[0][0]] == 2 or labels1[zz[0][0]] == 6:
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Flood - Low - Crop Stage 3')
    relief_fund=total_affected_region*350
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
    # --------- NO FLOOD
  elif labels1[zz[0][0]] == 3 :
    print('-------------------------')
    print()
```

```
    st.write('Identified Flood - No')
    # --------- FLOOD SEVERITY
  elif labels1[zz[0][0]] == 7:
    print('------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Paddy Flood Severity Level - Extreme Flood')
    relief_fund=total_affected_region*350
    st.write('------------------------')
    st.write('Relief Fund : ',relief_fund)
  elif labels1[zz[0][0]] == 8:
    print('------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Paddy Flood Severity Level - Flood + Cyclone')
    relief_fund=total_affected_region*300
    st.write('------------------------')
    st.write('Relief Fund : ',relief_fund)
  elif labels1[zz[0][0]] == 9:
    print('------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
```

```
    st.write('Identified Paddy Flood Severity Level - Low Flood')
    relief_fund=total_affected_region*50
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
elif labels1[zz[0][0]] == 10:
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Paddy Flood Severity Level - No Flood')
    # ---------- CYCLONE
elif labels1[zz[0][0]] == 11:
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Cyclone - Extreme')
    relief_fund=total_affected_region*300
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
elif labels1[zz[0][0]] == 12:
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
```

```
    st.write('Identified Cyclone - Low ')
    relief_fund=total_affected_region*50
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
elif labels1[zz[0][0]] == 13:
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Cyclone - No ')
    # ---------- DROUGHT
elif labels1[zz[0][0]] == 14:
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Drought - Extreme')
    relief_fund=total_affected_region*300
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
elif labels1[zz[0][0]] == 15:
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
```

```
st.write('Identified Drought - Low ')
relief_fund=total_affected_region*50
st.write('-------------------------')
st.write('Relief Fund : ',relief_fund)


elif labels1[zz[0][0]] == 16:
  print('-------------------------')
  print()
  st.write("Total affected region:", total_affected_region)
  st.write('Identified Drought - No ')
except:
  test_data=[]
  HEIGHT = 55
  WIDTH = 55
  N_CHANNELS = 3
  test_image_o = cv2.imread("Original.png")
  test_image = cv2.resize(test_image_o, (WIDTH, HEIGHT))
  test_data.append(test_image)
  # scale the raw pixel intensities to the range [0, 1]
  test_data = np.array(test_image, dtype="float") / 255.0
  test_data=test_data.reshape([-1,55, 55, 3])
  from keras.models import load_model
```

```
loaded_model = load_model("full_model.h5")
categories = os.listdir('Dataset/')
from numpy import argmax
pred = loaded_model.predict(test_data)
predictions = argmax(pred, axis=1) # return to label
labelss= categories[predictions[0]]
print ('Prediction : '+categories[predictions[0]])
# --------- EXTREME and CROP STAGE 1
if labelss == 'Extreme Flood' or labelss=='Crop Stage 1':
    print('------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Flood - Extreme - Crop Stage 1')
    relief_fund=total_affected_region*200
    st.write('------------------------')
    st.write('Relief Fund : ',relief_fund)
elif labelss == 'Extreme Flood' or labelss=='Crop Stage 2':
    print('------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Flood - Extreme - Crop Stage 2')
    relief_fund=total_affected_region*300
```

```
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
elif labelss == 'Extreme Flood' or labelss=='Crop Stage 3':
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Flood - Extreme - Crop Stage 3')
    relief_fund=total_affected_region*400
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
   # --------- LOW and CROP STAGE 2
elif labelss =='Low Flood' or labelss=='Crop Stage 1':
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Flood - Low - Crop Stage 1')
    relief_fund=total_affected_region*150
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
elif labelss =='Low Flood' or labelss=='Crop Stage 2' :
    print('-------------------------')
    print()
```

```
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Flood - Low - Crop Stage 2')

    relief_fund=total_affected_region*250
    st.write('--------------------------')
    st.write('Relief Fund : ',relief_fund)

elif labelss =='Low Flood' or labelss=='Crop Stage 3' :
    print('--------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Flood - Low - Crop Stage 3')
    relief_fund=total_affected_region*350
    st.write('--------------------------')
    st.write('Relief Fund : ',relief_fund)
     # --------- NO FLOOD
elif labelss == 'No Flood' :
    print('--------------------------')
    print()
    st.write('Identified Flood - No')
     # --------- FLOOD SEVERITY
elif labelss == 'Se_Extreme Flood':
```

```
    print('------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Paddy Flood Severity Level - Extreme Flood')
    relief_fund=total_affected_region*350
    st.write('------------------------')
    st.write('Relief Fund : ',relief_fund)
  elif labelss == 'Flood + Cyclone':
    print('------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Paddy Flood Severity Level - Flood + Cyclone')
    relief_fund=total_affected_region*300
    st.write('------------------------')
    st.write('Relief Fund : ',relief_fund)
  elif labelss == 'Se_Low Flood':
   print('------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Paddy Flood Severity Level - Low Flood')
    relief_fund=total_affected_region*50
    st.write('------------------------')
```

```
    st.write('Relief Fund : ',relief_fund)
elif labelss == 'Se_No Flood':
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Paddy Flood Severity Level - No Flood')
    # ---------- CYCLONE
elif labelss == 'Extreme Cyclone':

    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Cyclone - Extreme')
    relief_fund=total_affected_region*300
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
elif labelss == 'Low Cyclone':
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Cyclone - Low ')
    relief_fund=total_affected_region*50
```

```
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
elif labelss == 'No Cyclone':
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Cyclone - No ')
     # ---------- DROUGHT
elif labelss == 'Extreme Drought':
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Drought - Extreme')
    relief_fund=total_affected_region*300
    st.write('-------------------------')
    st.write('Relief Fund : ',relief_fund)
elif labelss == 'Low Drought':
    print('-------------------------')
    print()
    st.write("Total affected region:", total_affected_region)
    st.write('Identified Drought - Low ')
```

CHAPTER 9

SCREENSHOTS

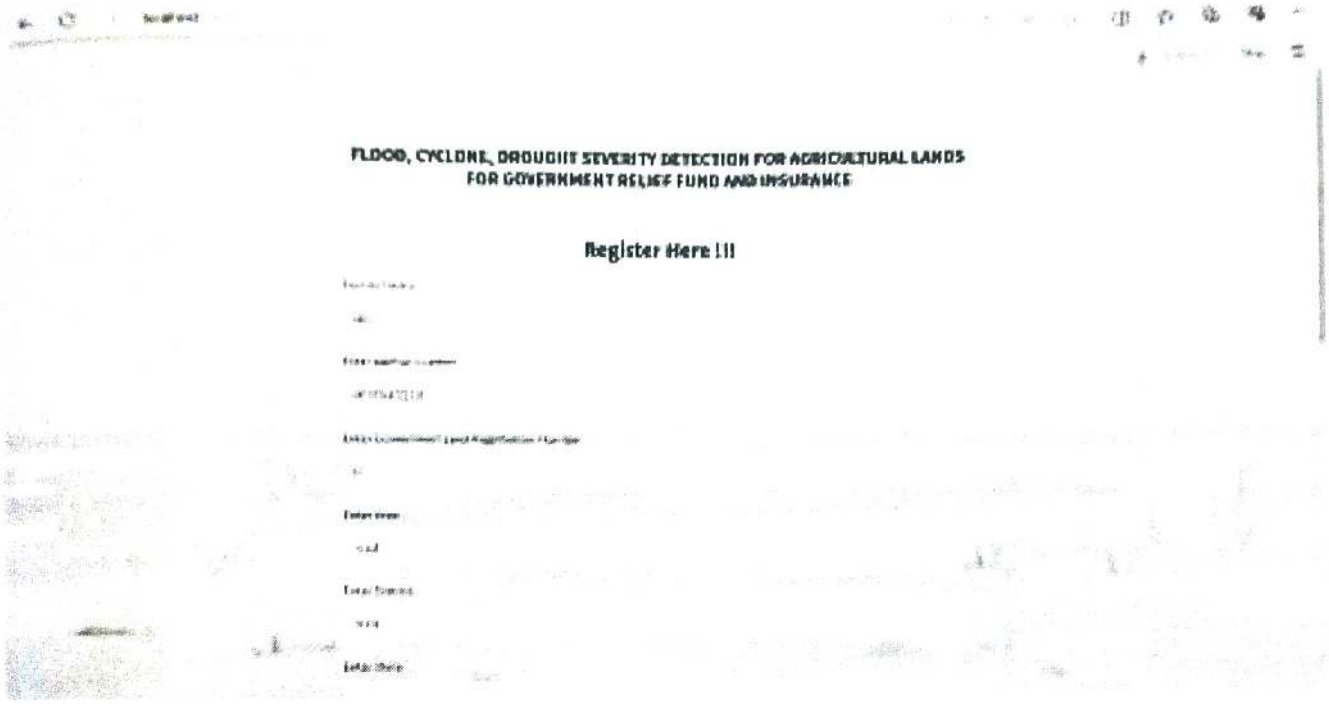


Fig 01 Register Page

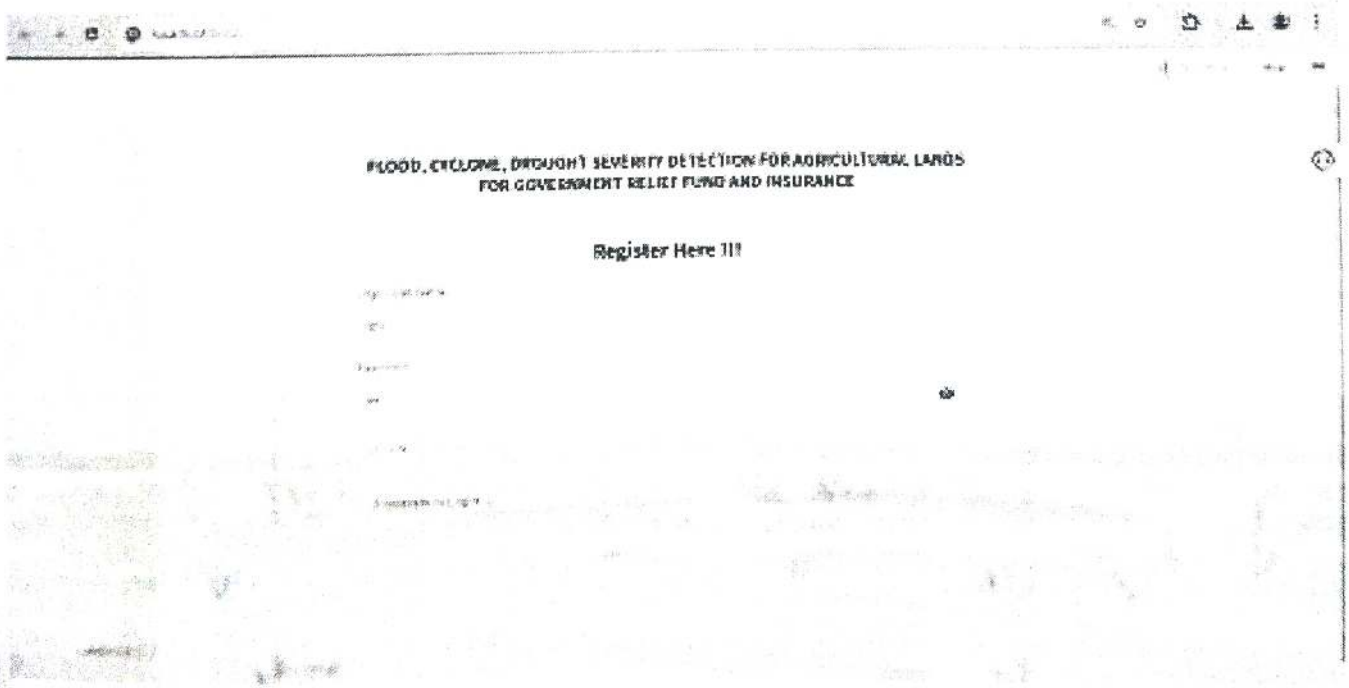


Fig 02 Login Page

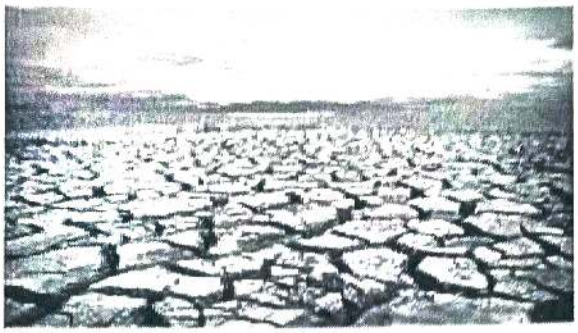


Fig 03 Input Image

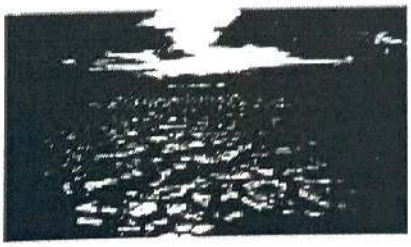


Fig 04 Gray Scale Conversion



Fig 05 Performance Analysis

Fig 06 Accuracy



Fig 07 Feature Extraction

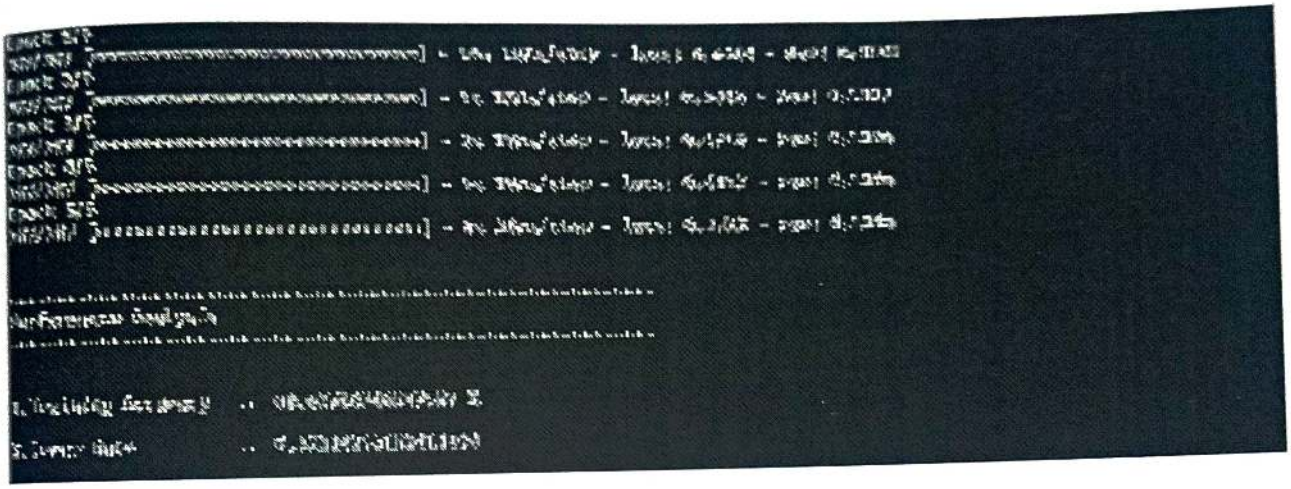


Fig 08 Accuracy in console

CHAPTER 10

REFERENCES

1. Smith, A. B., & Katz, R. W. (2013). US billion-dollar weather and climate disasters: Data sources, trends, accuracy and biases. Natural Hazards, 67(2), 387-410.
2. Mishra, A. K., & Singh, V. P. (2010). A review of drought concepts. Journal of Hydrology, 391(1-2), 202-216.
3. Zhang, X., Srinivasan, R., Lohani, V., & Lamb, M. (2005). Comparison of regression and neural network methods for runoff and sediment yield modeling. Environmental Modelling & Software, 20(3), 395-405.
4. Rahman, M. S., Zhao, X., & Mishra, A. K. (2019). Flood risk assessment using remote sensing and GIS: A review. Water, 11(5), 927.
5. Niyogi, D., Chang, H. I., & Krishnamurti, T. N. (2019). Climate dynamics of tropical cyclone-induced floods: A review. Journal of Hydrometeorology, 20(6), 1045-1063.
6. Ward, P. J., Jongman, B., Kummu, M., Dettinger, M. D., & Winsemius, H. C. (2019). Strong influence of El Niño Southern Oscillation on flood risk around the world. Proceedings of the National Academy of Sciences, 116(38), 19673-19678.
7. Sivakumar, M. V. K., & Stefanski, R. (2011). Climate change in the Asia-Pacific region: A call for action. International Journal of Climatology, 31(7), 851-854.
8. World Bank. (2016). Shock Waves: Managing the Impacts of Climate Change on Poverty. Washington, DC: World Bank.
9. FAO. (2017). The State of Food and Agriculture 2017: Leveraging Food Systems for Inclusive Rural Transformation. Rome: Food and Agriculture Organization of the United Nations.

10. IPCC. (2014). Climate Change 2014: Impacts, Adaptation, and Vulnerability. Part A: Global and Sectoral Aspects. Contribution of Working Group II to the Fifth Assessment Report of the Intergovernmental Panel on Climate Change [Field, C. B., et al. (Eds.)]. Cambridge, UK and New York, NY, USA: Cambridge University Press.
11. Hossain, F., Ferdous, M. R., & Rahman, M. S. (2018). Assessment of flood hazard, vulnerability and risk of Bangladesh. Environmental Science and Pollution Research, 25(15), 14872-14887.
12. Mishra, A. K., & Singh, V. P. (2010). A review of drought concepts. Journal of Hydrology, 391(1-2), 202-216.
13. Kala, J., & Andimuthu, R. (2014). Evaluating the performance of TRMM-based Multi-satellite Precipitation Analysis (TMPA) in driving a hydrological model for a large river basin in India. Journal of Hydrology, 517, 1011-1019.
14. Ahilan, S., & Chockalingam, L. (2011). Flood hazard modeling in lower Klang river basin using hydrodynamic approach. Water Resources Management, 25(8), 2005-2023.
15. IPCC. (2012). Managing the Risks of Extreme Events and Disasters to Advance Climate Change Adaptation. A Special Report of Working Groups I and II of the Intergovernmental Panel on Climate Change [Field, C. B., et al. (Eds.)]. Cambridge, UK and New York, NY, USA: Cambridge University Press.
16. Hirabayashi, Y., Mahendran, R., Koirala, S., Konoshima, L., Yamazaki, D., Watanabe, S., ... & Kanae, S. (2013). Global flood risk under climate change. Nature Climate Change, 3(9), 816-821.
17. Merz, B., Kreibich, H., Schwarze, R., & Thieken, A. H. (2010). Review article 'Assessment of economic flood damage'. Natural Hazards and Earth System Sciences, 10(8), 1697-1724.

18. Arnell, N. W. (2004). Climate change and global water resources: SRES emissions and socio-economic scenarios. Global Environmental Change, 14(1), 31-52.

19. Weisheimer, A., & Palmer, T. N. (2014). On the reliability of seasonal climate forecasts. Journal of the Royal Society Interface, 11(96), 20131162.

20. FAO. (2015). Climate Change and Food Security: Risks and Responses. Rome: Food and Agriculture Organization of the United Nations.